

Jackett-in-the-Lava-Local-Stack — Implementation (Phase 1)

Status: **first concrete, testable slice landed** (Torznab XML client in Go + compose service definition). No container booted; no commit/push. Author: implementation agent, 2026-06-08. Companion research dossier: [jackett-local-stack-research.md](#). Operator decision implemented: **Jackett sidecar in the Lava local stack, fronted by lava-api-go**.

This document describes (Part 1) the Torznab client that lava-api-go uses to talk to Jackett, and (Part 2) the compose service definition that runs Jackett (+ FlareSolverr) on the existing lava-net bridge.

Part 1 — Torznab XML client (lava-api-go/internal/jackett)

Files

File	Role
lava-api-go/internal/jackett/torznab.go	encoding/xml decoder + clean Result struct
lava-api-go/internal/jackett/client.go	Client (URL building, Search, Download with the 302→magnet handler)
lava-api-go/internal/jackett/torznab_test.go	Unit tests with real field assertions
lava-api-go/internal/jackett/testdata/torznab_results.xml	Hand-authored realistic Torznab feed (one .torrent item + one magnet item)

What it parses

A Torznab results feed is RSS 2.0 with the extension namespace `xmlns:torznab="http://torznab.com/schemas/2015/feed"`. Each `<item>` carries:

- `<title>`, `<guid>`,
- `<enclosure url=... length=... type=...>` — the download link, where type is `application/x-bittorrent` (a .torrent file) or `application/x-bittorrent;x-scheme-handler/magnet` (a magnet), and

- `<torznab:attr name=... value=.../>` entries — we extract seeders, size, magneturl, infohash.

These map to the exported `jackett.Result`:

```
type Result struct {
    Title           string
    GUID            string
    DownloadURL     string // <enclosure url>
    EnclosureType   string // EnclosureTypeTorrent |
                        EnclosureTypeMagnet
    Seeders         int    // -1 == unknown (distinguishes absent
                        from a real 0)
    Size            int64 // size attr, falling back to <enclosure
                        length>
    MagnetURL       string // magneturl attr, or the enclosure URL
                        if it's a magnet
    Infohash        string // lower-cased
}
```

Parsing notes that matter for correctness:

- **Seeders defaults to -1** so "no seeders attribute" is not silently reported as "0 seeders" (a download-confirmation lie).
- **Size falls back to <enclosure length>** when no size attr is present.
- **Infohash is lower-cased** (feeds emit upper-case; consumers expect lower).
- The decoder matches on the XML local name `attr`, so it also catches the `newznab:attr` spelling some indexers emit.

URL building (§6.R — nothing hardcoded)

`Client` is constructed from a `jackett.Config{ BaseURL, APIKey, Timeout }`. Both `BaseURL` and `APIKey` are **injected**, never literals (see Part 2 for where they come from). `BuildSearchURL` produces the documented Torznab path:

```
<base>/api/v2.0/indexers/<indexerID>/results/torznab/api?
apikey=<key>&t=search&q=<query>
```

`BuildCapsURL` produces the `t=caps` variant used as the readiness probe. The Torznab path segments themselves (`/api/v2.0/...`, `t`, `search`, `caps`) are protocol constants, not deployment addresses — they live as package constants, which is consistent with §6.R (the rule targets connection addresses, ports, and secrets, all of which live in `Config`).

The 302 → magnet edge case (handled, not auto-followed)

`Jackett` answers a `download//dl/` link with **HTTP 302 whose Location is a magnet: URI**. A naive HTTP client that auto-follows redirects would try to "GET magnet:" and fail. `jackett.Client` therefore configures `http.Client.CheckRedirect = http.ErrUseLastResponse` (do NOT follow), and `Download` captures the `Location` header verbatim:

```

dr, _ := c.Download(ctx, item.DownloadURL)
if dr.IsMagnet() { use(dr.Magnet) } else {
    writeTorrent(dr.TorrentBytes) }

```

Download returns DownloadResult{ Magnet | TorrentBytes }:

- enclosure already a magnet → short-circuits, no HTTP round-trip;
- HTTP 302 with Location: magnet:... → captured into Magnet;
- HTTP 302 with a non-magnet Location → explicit error (don't follow blindly);
- HTTP 200 → body returned as TorrentBytes + ContentType.

This directly serves the worklog's download-confirmation goal (valid .torrent bytes OR a real magnet) and the known kinozal/nnmclub null-magnet gaps.

Tests + verbatim go test output

The unit tests assert parsed fields against the hand-authored fixture (both an enclosure-.torrent item AND a magneturl/infohash item), the -1-vs-0 seeders distinction, URL building from config, and the 302→magnet handler (via httptest).

Command:

```
cd lava-api-go && go test ./internal/jackett/... -run . -count=1
```

Verbatim output:

```
ok      digital.vasic.lava.apigo/internal/jackett    0.267s
```

Verbose run (per-test):

```

=== RUN   TestParseResults_TorrentEnclosureItem
--- PASS: TestParseResults_TorrentEnclosureItem (0.00s)
=== RUN   TestParseResults_MagnetItem
--- PASS: TestParseResults_MagnetItem (0.00s)
=== RUN   TestParseResults_SeedersUnknownVsZero
--- PASS: TestParseResults_SeedersUnknownVsZero (0.00s)
=== RUN   TestParseResults_EmptyFeed
--- PASS: TestParseResults_EmptyFeed (0.00s)
=== RUN   TestParseResults_MalformedXML
--- PASS: TestParseResults_MalformedXML (0.00s)
=== RUN   TestBuildSearchURL
--- PASS: TestBuildSearchURL (0.00s)
=== RUN   TestBuildCapsURL
--- PASS: TestBuildCapsURL (0.00s)
=== RUN   TestNewClient_MissingConfig
--- PASS: TestNewClient_MissingConfig (0.00s)
=== RUN   TestDownload_302ToMagnet
--- PASS: TestDownload_302ToMagnet (0.00s)
=== RUN   TestDownload_TorrentFile
--- PASS: TestDownload_TorrentFile (0.00s)
=== RUN   TestDownload_MagnetEnclosureShortCircuits
--- PASS: TestDownload_MagnetEnclosureShortCircuits (0.00s)
=== RUN   TestSearch_EndToEnd

```

```
ok      digital.vasic.lava.apigo/internal/jackett    0.368s
```

Part 2 — Local-stack compose service definition

tools/lava-containers/docker-compose.jackett.yml — a **compose fragment** that adds lava-jackett (+ optional lava-flaresolverr) to the Lava stack. It is a fragment (not edits to the root docker-compose.yml) so it composes in via -f and the root file keeps owning the bridge + the api-go/observability profiles.

```
Android app —mDNS(_lava-api._tcp)→ lava-api-go (:8443, host-  
net, TLS+HMAC)  
  
                                | (server-side HTTP,  
holds api_key)  
  
                                ▼  
                                lava-jackett (:9117) —  
  
lava-net, NOT advertised  
  
                                |  
                                (only when profile=cloudflare)  
  
                                ▼  
                                lava-flaresolverr (:8191) —  
  
lava-net
```

Service	Image (digest-pin in .env)	Network	Port	Profile	Notes
lava-jackett	lscr.io/linuxserver/jackett (multi-arch amd64+arm64)	lava-net (bridge)	9117 internal only (expose, no ports)	jackett, cloudflare	/config = gitignored host volume holding ServerConfig.json (api_key) + indexer creds
lava-flaresolterr	ghcr.io/flaresolterr/flaresolterr	lava-net	8191 internal only	cloudflare	RAM-heavy headless Chromium; up only when IPTorrents (or other CF tracker) is enabled

Ports exposed to the LAN: none

Neither 9117 nor 8191 is published. They use `expose:` (intra-bridge only). Only `lava-api-go:8443` is LAN-visible. This is a security requirement — an unauthenticated Jackett admin surface on the LAN is rejected by the dossier §7.

Who holds the apikey + how lava-api-go consumes it

- **Jackett generates the `api_key` on first run** and persists it in the gitignored `/config/ServerConfig.json` host volume (`${LAVA_JACKETT_CONFIG_DIR}` → `/config`).
- **lava-api-go reads it server-side at startup** (env `LAVA_JACKETT_API_KEY`, populated by the Containers glue from that host file) and constructs a `jackett.Config{ BaseURL: "http://lava-jackett:9117", APIKey: <key> }`.
- The `api_key` is a **§6.H secret**: server-side only, **never** hardcoded (§6.R), **never** sent to the device. The app talks only to `lava-api-go`, which proxies Torznab. The app never sees Jackett directly.

Readiness healthcheck (§6.B-correct — not a bare TCP probe)

- **Jackett**: a Torznab `t=caps` probe — `wget http://127.0.0.1:9117/api/v2.0/indexers/<id>/results/torznab/api?t=caps&apikey=<key>`. A 200 + non-empty body proves the Torznab surface is live AND the `api_key` is valid — the same code path `lava-api-go` consumes. A bare TCP-9117 or a 200 on / would be a bluff probe (§6.B: "Up" != application-healthy). `start_period: 40s` covers the .NET cold-start (UNCONFIRMED seconds per the dossier — tighten once measured on the target host).
- **FlareSolvrr**: `POST /v1 {"cmd":"sessions.list"} → 200` (no `/health` endpoint exists; a bare TCP-8191 check would accept the Python proxy before Chromium is ready).

§6.A follow-up (owed when wiring the actual healthprobe into the gate): if a Lava-owned healthprobe-style binary is ever used for the Jackett probe instead of in-container `wget`, it **MUST** gain a contract test asserting its flag set, mirroring `lava-api-go/tests/contract/healthcheck_contract_test.go`.

§6.R env inventory (placeholders belong in `.env.example`; real values in gitignored `.env`)

Variable	Purpose	Default in fragment
<code>LAVA_JACKETT_IMAGE</code>	Jackett image, pin by digest	<code>lscr.io/linuxserver/jackett:latest</code>
<code>LAVA_FLARESOLVERR_IMAGE</code>	FlareSolvrr image, pin by digest	<code>ghcr.io/flaresolvrr/flaresolvrr:latest</code>
<code>LAVA_JACKETT_CONFIG_DIR</code>	host path for / config	<code>./.jackett-config</code>

Variable	Purpose	Default in fragment
	(gitignored secrets store)	
LAVA_JACKETT_API_KEY	api_key read from /config (healthcheck + lava-api-go)	(required — no default)
LAVA_JACKETT_HEALTH_INDEXER	indexer id used by the caps probe	all
LAVA_JACKETT_PUID / _PGID / _TZ	LinuxServer user/group/ timezone	1000 / 1000 / Etc/UTC
LAVA_FLARESOLVERR_LOG_LEVEL	FlareSolvert log level	info

These `.env` / `.env.example` keys are **owed** to the parent `.env.example` when this slice is wired into orchestration; this phase touches only the `compose/tools/jackett-doc` surface per scope, so the variables are documented here and defaulted in the fragment rather than added to `.env.example` yet.

Parse verification (no boot)

```
cd tools/lava-containers
LAVA_JACKETT_API_KEY=placeholder podman compose -f docker-
compose.jackett.yml config --quiet # → parses clean
```

No container was started; orchestration remains owned by `tools/lava-containers/cmd/lava-containers`. The fragment is merged with the root `compose` at run time, e.g.:

```
docker compose -f docker-compose.yml -f tools/lava-containers/
docker-compose.jackett.yml \
  --profile api-go --profile jackett config
```

What is intentionally NOT in this phase

- No indexer provisioning automation (dossier Open Question Q3 — admin HTTP vs per-/config JSON; both need a §6.A contract test + pinned Jackett version).
- No `lava-api-go` route/provider wiring (the `jackett.Client` exists and is tested; hooking it into a handler + the OpenAPI spec is the next slice).
- No `.env` / `.env.example` edits (out of this phase's scope; variables inventoried above).
- No container boot, no commit, no push.